

Decoder-Only Transformers: Building GPT From Scratch

Objective

In Task 1, you explored contextual representations through BERT and learned how Transformer encoders understand language.

In Task 2, you explored language modelling through N-grams and sequence labelling through Hidden Markov Models and BERT-based POS tagging.

In this task, you will build a Decoder-Only Transformer (GPT) from scratch and investigate how modern Large Language Models perform next-token prediction and text generation.

This task is designed to bridge:

Bigram Language Models



Hidden Markov Models



Attention



GPT-style Language Models

By the end of this assignment, you should understand:

- Why Markov assumptions fail
- How self-attention captures long-range dependencies
- Why decoder-only transformers dominate modern NLP
- How next-token prediction enables language generation
- The conceptual relationship between HMM decoding and autoregressive generation

Phase I: Revisiting Language Models

Before implementing GPT, revisit language modelling from first principles.

Given:

The cat sat on the

a language model estimates:

$P(\text{next_token} \mid \text{previous_tokens})$

Tasks

Implement:

1. Unigram Language Model
2. Bigram Language Model
3. Trigram Language Model

Use:

- Add-1 Smoothing
- Perplexity Evaluation

Dataset:

- Tiny Shakespeare
- WikiText-2
- The Office Corpus

Deliverables

ngram_lm.py

Concept Questions

1. Why does perplexity decrease as context size increases?
2. Why do N-gram models eventually become impractical?
3. What assumptions are made by a Bigram Model?

Phase II: Subword Tokenization

Modern language models do not operate on words directly.

Implement a simplified Byte Pair Encoding (BPE) tokenizer.

Tasks

1. Start with character vocabulary
2. Learn merge rules
3. Construct subword vocabulary
4. Encode text into subword tokens

Compare:

- Word-level tokenization
- Character-level tokenization
- BPE tokenization

Analysis

For each method report:

- Vocabulary size
- OOV behaviour
- Average sequence length

Deliverables

bpe_tokenizer.py

Concept Questions

1. Why do GPT models use subword tokenization?
2. How does BPE help with rare words?
3. Why are word-level vocabularies problematic?

Phase III: Positional Information

Self-attention alone has no notion of order.

Implement positional representations.

Tasks

Implement:

1. Sinusoidal Positional Encoding
2. Learned Positional Embeddings

Compare both approaches.

Deliverables

positional_encoding.py

Analysis

Visualize positional vectors.

Answer:

1. Why does attention require positional information?
2. What advantages do learned embeddings provide?

Phase IV: Scaled Dot Product Attention

Implement self-attention from scratch.

Given:

$$Q = XW_q$$

$$K = XW_k$$

$$V = XW_v$$

Compute:

Attention(Q,K,V)

=

$$\text{softmax}(QK^T / \sqrt{d})V$$

Restrictions

Do not use:

nn.MultiheadAttention

Deliverables

attention.py

Analysis

Visualize attention scores on sample sentences.

Phase V: Causal Masking

GPT must not observe future tokens.

Implement a causal mask.

Example

When predicting:

sat

from:

The cat sat

the model must not see:

sat

during prediction.

Tasks

1. Implement triangular attention mask
2. Visualize masked attention matrix

Deliverables

masking.py

Concept Questions

1. Why does GPT require masking?
2. Why does BERT not use masking in this way?

Phase VI: Multi-Head Attention

Extend self-attention into Multi-Head Attention.

Tasks

Implement:

```
class MultiHeadAttention
```

Steps:

1. Split embedding dimensions
2. Compute attention independently
3. Concatenate outputs
4. Final projection

Deliverables

multihead.py

Analysis

Visualize attention maps from different heads.

Discuss whether different heads learn different behaviours.

Phase VII: Decoder Transformer Block

Construct a full GPT decoder block.

Architecture:

Input

↓

Masked Multi-Head Attention

↓

Add + LayerNorm

↓

Feed Forward Network

↓

Add + LayerNorm

↓

Output

Requirements

Implement:

- Residual Connections
- LayerNorm
- Feed Forward Network
- Dropout

Deliverables

decoder_block.py

Phase VIII: MiniGPT

Construct a complete GPT-style language model.

Architecture:

Token Embeddings

+

Positional Embeddings

↓

N Decoder Blocks

↓

Linear Projection

↓

Vocabulary Logits

Recommended Configuration:

layers = 4

heads = 4

embedding_dim = 128

context_length = 128

Deliverables

gpt.py

Phase IX: Training

Train MiniGPT on one dataset.

Recommended:

- Tiny Shakespeare
- WikiText-2
- The Office Corpus

Tasks

1. Build training sequences
2. Implement next-token prediction
3. Train using Cross Entropy Loss
4. Monitor loss and perplexity

Deliverables

train.py

Phase X: Text Generation

Implement multiple decoding strategies.

Required

1. Greedy Decoding
2. Temperature Sampling
3. Top-k Sampling
4. Top-p Sampling (Nucleus Sampling)

Optional

Beam Search

Tasks

Generate text from prompts.

Example:

Michael:

Dwight:

The future of AI is

Compare outputs.

Deliverables

generate.py

Analysis

Discuss:

- Creativity
- Repetition
- Coherence
- Diversity

Phase XI: Evaluation

Evaluate:

Language Modelling

Report:

- Training Perplexity
- Validation Perplexity

Generation Quality

Generate at least:

10 samples

Comment on:

- Grammar
- Fluency
- Consistency
- Hallucination

Phase XII: Attention Analysis

Choose generated samples.

Visualize attention matrices.

For each head discuss:

1. Local dependencies
2. Long-range dependencies
3. Structural patterns

Deliverables

attention_visualization.py

Phase XIII: Error Analysis and Comparisons

Answer all questions in README.md.

Part A: Bigram vs GPT

Find 3 examples where:

GPT succeeds and Bigram fails.

Explain:

- Context length
- Dependency modelling
- Vocabulary generalisation

Part B: HMM vs GPT

Compare:

Property	HMM	GPT
Context Size		
Markov Assumption		

Training Objective

Long-Range
Dependencies

Scalability

Part C: BERT vs GPT

Compare:

Property	BERT	GPT
Architecture		
Attention Type		
Training Objective		
Generation Capability		
Typical Applications		

Part D: Viterbi vs Autoregressive Decoding

Explain:

1. How Viterbi searches for globally optimal tag sequences
2. How autoregressive decoding generates sequences token by token
3. Why Beam Search can be viewed as a loose analogue of Viterbi search

Part E: Reflection

Answer:

Why does next-token prediction lead to language understanding?

This should be answered thoughtfully and in your own words.

Papers to Read

Required

Attention Is All You Need
(Vaswani et al., 2017)

Focus on:

- Multi-Head Attention
- Positional Encoding
- Decoder Architecture

Required

Language Models are Unsupervised Multitask Learners
(Radford et al., GPT-2)

Focus on:

- Autoregressive Language Modelling
- Emergent Behaviour

Recommended

The Illustrated Transformer
(Jay Alammar)

Submission Requirements

File	Description
ngram_lm.py	Unigram, Bigram, Trigram Models
bpe_tokenizer.py	Subword Tokenization

<code>positional_encoding.py</code>	Positional Representations
<code>attention.py</code>	Self-Attention
<code>masking.py</code>	Causal Masking
<code>multihead.py</code>	Multi-Head Attention
<code>decoder_block.py</code>	Decoder Block
<code>gpt.py</code>	Complete MiniGPT
<code>train.py</code>	Training Pipeline
<code>generate.py</code>	Text Generation
<code>attention_visualization.py</code>	Attention Analysis
<code>README.md</code>	Concept Questions and Error Analysis

Stretch Goals

Choose one:

1. KV-Caching
2. Flash Attention Study
3. Rotary Positional Embeddings (RoPE)
4. Tiny Instruction Tuning Dataset
5. GPT vs LSTM Language Modelling Comparison

These are intended for students interested in modern LLM research directions.